BIN702 - Série d'exercices #4 : l'alignement multiple

Manuel Lafond

Exercice 1: Soient les séquences

$$
\begin{aligned}
S_1 &= AGTA \\
S_2 &= AGCTA \\
S_3 &= TCA \\
S_4 &= TGA
\end{aligned}
$$

a. Considérez l'algorithme de centre-étoile vu en classe, avec un coût de 1 pour toute mutation et tout caractère aligné avec un gap (donc un gap avec un gap a un coût de 0). Quelle serait une séquence centrale possible ? Donnez le résultat de l'algorithme selon la séquence centrale choisie.

   Solution.

   La figure suivante illustre la sélection de $S_c = S_1$ selon la somme de ses distances dans la matrice (on suppose ici que je ne fais aucune erreur !). La construction de l'alignement centré à S_1 est aussi illustré.

	S_1	S_2	S_3	S_4
S_1	0	1	3	2
S_2	1	0	3	3
S_3	3	3	0	1
S_4	2	3	1	0

Choix: $S_c = S_1$

Étape 1:

S_1 A G - T A
S_2 A G C T A

Étape 2

S_1 A G - T - A
S_2 A G C T - A
S_3 - - - T C A

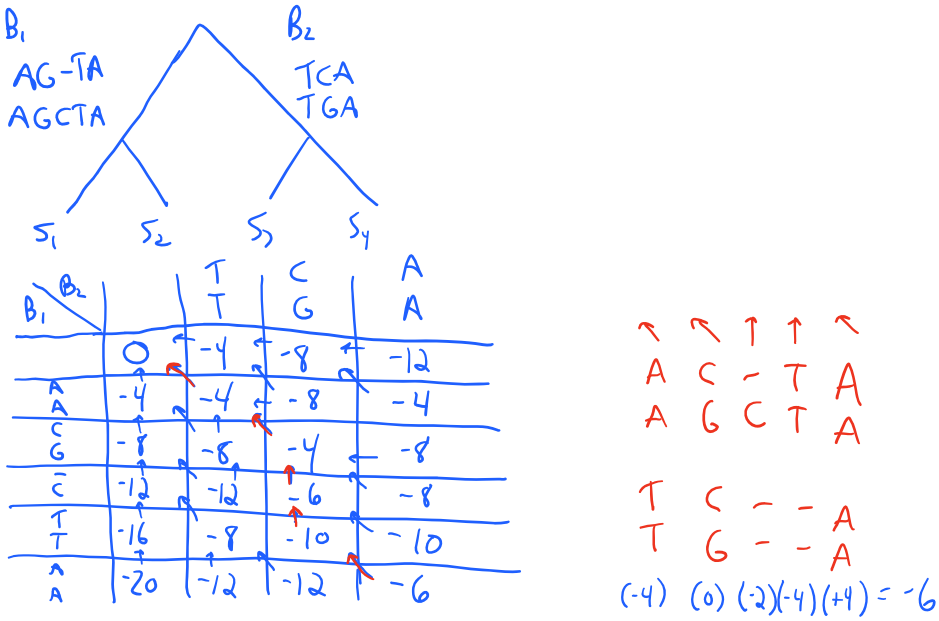
Étape 3

S_1 A G - T - A
S_2 A G C T - A
S_3 - - - T C A
S_4 T G - - - A

b. Considérez l'algorithme de Clustal pour maximiser le score SP, avec un score de $+1$ pour un match, -1 pour une mutation ou un caractère aligné avec un gap, et 0 pour un gap avec un gap. Supposez que vous avez l'arbre guide dans lequel S_1 et S_2 ont un parent commun, et S_3 et S_4 ont un autre parent commun (et ces deux parents sont joints par la racine).

   Donnez l'alignement multiple retourné par Clustal suite au parcours de l'arbre guide. Pour bien faire cet exercice, vous devriez construire la table de programmation dynamique pour combiner les deux alignements du 2ème niveau de l'arbre guide.

Solution.
La table de programmation dynamique pour le niveau de la racine est donné dans la figure suivante. Un alignement donné par Clustal est donné à droite.



Exercice 2: Écrivez les conditions initiales et la récurrence servant à calculer l'alignement multiple optimal entre 3 séquences S_1, S_2 et S_3. Quelle est la complexité de l'algorithme sous-jacent qui calcule la table de programmation dynamique.

Solution.

Soit M la matrice de scores. On suppose $M[-,-] = 0$. Soit $V[i,j,k]$ le score d'alignement optimal entre $S_1[1..i], S_2[1..j]$ et $S_3[1..k]$.

On a $V[0,0,0] = 0$. On définit $V[i,j,k] = -\infty$ si on a $i < 0, j < 0$ ou $k < 0$. Pour $i,j,k \geq 0$ tels que $i+j+k > 0$, on doit énumérer toutes les façons d'avoir la dernière colonne de l'alignement multiple, en excluant le cas où il y a un gap partout.

$$V[i,j,k] = \max \begin{cases} V[i-1,j,k] + M[S_1[i],-] + M[S_1[i],-] \\ V[i,j-1,k] + M[S_2[j],-] + M[S_2[j],-] \\ V[i,j,k-1] + M[S_3[k],-] + M[S_3[k],-] \\ V[i-1,j-1,k] + M[S_1[i],S_2[j]] + M[S_1[i],-] + M[S_2[j],-] \\ V[i-1,j,k-1] + M[S_1[i],S_3[k]] + M[S_1[i],-] + M[S_3[k],-] \\ V[i,j-1,k-1] + M[S_2[j],S_3[k]] + M[S_2[j],-] + M[S_3[k],-] \\ V[i-1,j-1,k-1] + M[S_1[i],S_2[j]] + M[S_1[i],S_3[k]] + M[S_2[j],S_3[k]] \end{cases}$$

Exercice 3: La récurrence utilisée par Clustal pour aligner deux blocs d'alignements multiple est

$$V[i,j] = max \begin{cases} V[i-1,j-1] + score(\text{colonne i}, \text{colonne j}) \\ V[i-1,j] + score(\text{colonne i}, -) \\ V[i,j-1] + score(-, \text{colonne j}) \end{cases}$$

Par paresse, je n'ai pas détaillé les fonctions de score (ou, devrais-je dire, j'ai gardé le score générique car Clustal en implémente plusieurs). Dans le contexte où on veut maximiser le score SP, écrivez précisément ce qu'on calcule pour les *scores*. Donnez ensuite la complexité de l'algorithme qui calcule la table de programmation dynamique.

Solution.

Dans le cas du score SP, on veut maximiser la somme des scores entre les paires de caractères dans les deux colonne jointes. Si M est la matrice de scores, soient $S_1, S_2, \ldots, S_r$ les séquences du premier bloc à aligner, et soient $T_1, T_2, \ldots, T_s$ les séquences du deuxième bloc à aligner. Alors

$$score(\text{colonne i}, \text{colonne j}) = \sum_{a=1}^{r} \sum_{b=1}^{s} M[S_a[i], T_b[j]]$$

$$score(\text{colonne i}, -) = \sum_{a=1}^{r} M[S_a[i], -]$$

$$score(-, \text{colonne j}) = \sum_{b=1}^{s} M[-, T_b[j]]$$

Chaque score prend un temps $O(r \cdot s)$ à calculer. Si les longeurs des blocs sont n et m, il y a nm cellules à calculer. la complexité totale devient $O(nm \cdot rs)$. Petite note, avec un peu d'effort, on peut atteindre $O(nm(r+s))$ si la taille de l'alphabet est constante.

Exercice 4 (Difficile)**:** Soit $S_1, \ldots, S_m$ un ensemble de m séquences. Soit A l'alignement multiple retourné par l'algorithme centre-étoile, et soit A^* l'alignement multiple optimal. Montrez que

$$cout(A) \leq 2cout(A^*)$$

L'inégalité triangulaire stipule que la matrice de coûts M satisfait $M[a,b] \leq M[a,c] + M[c,b]$ pour tout $a, b, c \in \Sigma \cup \{-\}$.

Plus précisément, si S_c est la séquence centrale, vous pouvez montrer que et que

$$cout(A) \leq m \sum_{i=1}^{m} D(S_c, S_i) \qquad \text{et} \qquad cout(A^*) \geq \frac{m}{2} \sum_{i=1}^{m} D(S_c, S_i)$$

où D est la distance minimum d'alignement global. La deuxième inégalité a été démontrée en classe. Pour la première, vous pouvez utiliser le fait que grâce à l'inégalité triangulaire, on a $dist_A(S_i, S_j) \leq dist_A(S_i, S_c) + dist_A(S_c, S_j) = D(S_c, S_i) + D(S_c, S_j)$.

Solution.
On va d'abord montrer que $cout(A) \leq m \sum_{i=1}^{m} D(S_c, S_i)$.
On a

$$\begin{aligned}
cout(A) &= \sum_{i=1}^{m} \sum_{j=i+1}^{m} dist_A(S_i, S_j) \\
&= \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} dist_A(S_i, S_j) \\
&\leq \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} (D(S_c, S_i) + D(S_c, S_j)) && \text{selon l'inégalité triangulaire} \\
&= \frac{1}{2} \left(\sum_{i=1}^{m} \sum_{j=1}^{m} D(S_c, S_i) + \sum_{i=1}^{m} \sum_{j=1}^{m} D(S_c, S_j) \right) \\
&= \frac{1}{2} \left(m \sum_{i=1}^{m} D(S_c, S_i) + m \sum_{j=1}^{m} D(S_c, S_j) \right) \\
&= \frac{1}{2} \left(2m \sum_{i=1}^{m} D(S_c, S_i) \right) \\
&= m \sum_{i=1}^{m} D(S_c, S_i)
\end{aligned}$$

Pour l'autre inégalité, on a

$$
\begin{aligned}
cout(A^*) &= \sum_{i=1}^{m} \sum_{j=i+1}^{m} dist_{A^*}(S_i, S_j) \\
&= \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} dist_{A^*}(S_i, S_j) \\
&\geq \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} D(S_i, S_j) && \text{car } D(S_i, S_j) \text{ est le minimum possible} \\
&\geq \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} D(S_c, S_j) && \text{par notre choix de } S_c \\
&= \frac{1}{2} m \sum_{j=1}^{m} D(S_c, S_j)
\end{aligned}
$$